

# Deep Code Extraction & Operational Audit Report

**Objective:** To perform an exhaustive line-by-line review, architectural mapping, and logical verification of the user-provided TensorFlow/RNN code execution pipeline, highlighting structural features, array dimensions, configuration mismatches, and mathematical behaviors.

**Maksad:** Jo RNN code aur output aapne share kiya hai, uske ek-ek hisse ka poora depth mein post-mortem karna. Data dimensions, framework ke hidden concepts, code ki configuration mistakes, aur output galat aane ki asli maths ko simple Hinglish bhasha mein ek professional report ke roop mein safe karna.

## 1. Structural Data Dimensions & Input Parsing

```
X_train = [  
    [1,2,3,4,5,6,7,8,9,10,11,12], # 12 elements  
    [1,2,13,4,14],                # 5 elements  
    [1,2,3,15],                   # 4 elements  
    [2,15,16,5,17],               # 5 elements  
    [2,15,18,19,20,21,22,23]      # 8 elements  
]  
x_train_padded = pad_sequences(X_train, maxlen=12, padding='pre')
```

### Technical Interpretation:

The input sequence represents 5 distinct tokenized documents with highly varying step lengths. Neural network input layers require uniform dense tensors. The `pad_sequences` function normalizes these jagged arrays into a unified rank-2 tensor of shape (5, 12) using zero-pre-padding.

### Bhasha Me Samjhein:

Hamare paas total 5 reviews hain, lekin sabki lambai (length) alag-alag thi—koi 12 words ka tha toh koi sirf 4 words ka. Neural network ko calculations karne ke liye ek perfect rectangular matrix chahiye hota hai. Isliye humne `pad_sequences` ka use kiya aur `maxlen=12` set kiya. Isne har sentence ke aage zeros (0) laga kar sabko barabar 12 ki length de di, jisse iska final structure (5, 12) ban gaya, yaani 5 rows aur 12 columns.

```
# Output metrics print values from runtime:  
<class 'numpy.ndarray'> (5, 12)  
<class 'list'> (5,)
```

The variable `Labels` was initially declared as a native Python list with a shape profile of `(5,)`. To make it compatible with backpropagation gradient computations in TensorFlow, it was correctly cast using `np.array(Labels)` to transition it into a structured NumPy array vector.

Shuruat mein hamari target values `Labels = [0,1,1,0,1]` ek normal Python list thi jiska type matrix math ke liye theek nahi tha. Isko code mein `Labels = np.array(Labels)` karke NumPy array mein badla gaya taaki TensorFlow ka loss calculation system smoothly chal sake.

## 2. Architectural Layer Matrix Breakdown & Hidden Flaws

The initialization pipeline stacks three core operational layers sequentially. However, a deep structural configuration audit reveals an explicit dimension mismatch between the tracking interfaces of Layer 1 and Layer 2.

Aapke model ke andar teen layers line se lagi hui hain, lekin jab hum iske parameters ka gahraai se audit karte hain, toh pehli aur doosri layer ke beech ek bohot bada architectural mismatch (galti) dikhta hai jo bina error aaye model ko kharab karta hai.

```
model.add(Embedding(input_dim=120, output_dim=8, input_length=100))
model.add(SimpleRNN(units=32, return_sequences=False))
```

### A. The Embedding Layer Dissection

- **input\_dim=120:** Defines the token range. The model allocates index spots for 120 unique integers.  
| Iska matlab model ke paas total 120 words ki dictionary space reserved hai.
- **output\_dim=8:** Compresses every integer token into a continuous spatial point containing 8 coordinate floats.  
| Yeh har word ko 8 alag-alag numbers ki list (dense vector) mein convert karta hai.
- **input\_length=100: [CRITICAL ARCHITECTURAL FLAW]** Explicitly tells the layer to expect an input stream sequence of exactly 100 timesteps.  
| **[Badi Galti]:** Yahan aapne model ko bola ki input mein hamesha 100 words aayenge!

#### Critical System Divergence (Dimension Disconnect)

**The Conflict:** Your `x_train_padded` array provides sequences of length **12** (`maxlen=12`). However, your Embedding layer configuration is explicitly hardcoded to parse a timeline length of **100** (`input_length=100`).

**Nuksan:** Jab aapne data ko pad kiya toh maximum length 12 thi, par layer ko aapne bola ki 100 length ka data aayega. Is mismatch ki wajah se network ke internal shapes block ho jate hain aur sequence tracking ki calculation kharab ho jati hai. Isko sahi karne ke liye `input_length=12` hona chahiye tha.

### B. The SimpleRNN Layer Internal Mapping

The SimpleRNN layer contains `units=32` tracking cells. With `return_sequences=False`, the step loop sequentially updates its internal hidden vector  $h_t$  at every timestep from index 1 to 12. Once the final timestep is processed, it completely discards the transient output sequence and emits only the final compressed memory array of shape `(Batch_Size, 32)`.

Hamare SimpleRNN ke paas 32 units ki memory hai. `return_sequences=False` lagane ki wajah se yeh layer har word par apna dimaag update toh karegi, lekin beech ke kisi step ka output nahi degi. Jab poora sentence khatam ho jayega, toh aakhiri 32 numbers ka vector utha kar aage bhej degi.

### 3. Optimization Convergence & Training Behavior

The compilation step binds the structural weights to optimization paths using the Adam variant and tracks errors via the binary cross-entropy metric expression.

Model ko compile karte waqt `loss='binary_crossentropy'` lagaya gaya kyunki hamara kaam Binary Classification (0 ya 1) ka hai. Optimization ke liye adam ka use kiya gaya hai jo weights ko dheere-dheere correct karta hai.

```
# Model execution loss log over 5 epochs:  
Epoch 1/5 -> loss: 0.6321 - accuracy: 0.8000  
Epoch 2/5 -> loss: 0.6188 - accuracy: 0.8000  
Epoch 3/5 -> loss: 0.6053 - accuracy: 0.8000  
Epoch 4/5 -> loss: 0.5922 - accuracy: 0.8000  
Epoch 5/5 -> loss: 0.5794 - accuracy: 0.8000
```

#### Mathematical Evaluation of Logs:

1. **Loss Reduction Trend:** The loss function decreases step-by-step from 0.6321 down to 0.5794. This proves that the backpropagation algorithm is functioning properly—weights are adapting, and errors are being minimized.

1. **Loss Ka Ghatna:** Loss lagatar har epoch mein kam ho raha hai (0.6321 se girkar 0.5794 tak pahoncha). Iska matlab model parde ke peeche seekhne ki koshish kar raha hai, gradients backpropagation ke zariye layers mein travel kar rahe hain.

2. **Stagnant Accuracy Phenomenon:** The training log shows a perfectly flat accuracy metric of exactly 0.8000 (80%) across all 5 epochs. This happens because out of your 5 total sentences, 4 are predicted correctly on the surface level, but the absolute probability scores are barely shifting away from the classification boundary line.

2. **Accuracy Ka Ek Jagah Rukna (80%):** Aapne notice kiya hoga ki accuracy har round mein exact 0.8000 par chipki hui hai. Aisa isliye hai kyunki total 5 records mein se 4 records boundary condition par sahi partition ho rahe hain, par model ka confidence abhi bohot dheere badh raha hai kyunki sirf 5 epochs ka mauka mila.

#### Full Training Prediction Matrix Output:

When you run `model.predict(x_train_padded)` right after training, the system evaluates all 5 training rows, outputting a 2D array matrix representation:

Jab poorani 5 training lines par hi prediction dobara check ki gayi, toh model ne sabka raw probability score nikaala:

| Review Index                | True Target Label | Model Probability Score Output | Hard Threshold Result (>0.5)      |
|-----------------------------|-------------------|--------------------------------|-----------------------------------|
| Review 1 (Negative Context) | 0                 | <b>0.4667</b>                  | 0 (Correct Classification)        |
| Review 2 (Positive Context) | 1                 | <b>0.6450</b>                  | 1 (Correct Classification)        |
| Review 3 (Positive Context) | 1                 | <b>0.6300</b>                  | 1 (Correct Classification)        |
| Review 4 (Negative Context) | 0                 | <b>0.5998</b>                  | 1 <b>(Failing False Positive)</b> |
| Review 5 (Positive Context) | 1                 | <b>0.6783</b>                  | 1 (Correct Classification)        |

## 4. Testing Phase Diagnostic & Failure Analysis

During the evaluation phase, you fed a completely unaligned token sequence array through the prediction pipeline, yielding an unexpected classification outcome:

Ab aate hain hamari testing phase par, jahan aapne ek naye token array ko pass kiya aur model ne galat output (False Positive) de diya:

```
new_review = [[1, 2, 3, 4, 5]]
# System output result received:
Prediction score for the new review is: 0.626953125
The new review is classified as positive.
```

### Why did the model classify this sequence as Positive?

To understand this failure, we must analyze the internal vocabulary mapping tokens from your training set. Look closely at how the tokens are distributed in your data:

Aakhir model ne ise Positive kyu bola jabki aapke mutabik sentence galat tha? Chalo token mappings ka poora calculations dekhte hain:

- In Review 1: "the movie is so bad..." mapped directly to [1, 2, 3, 4, 5...]. This means Token 1 = "the", Token 2 = "movie", Token 3 = "is", Token 4 = "so", Token 5 = "bad".  
| Aapke data ke hisab se tokens yeh hain: 1="the", 2="movie", 3="is", 4="so", 5="bad".
- Your test input array [ [1, 2, 3, 4, 5] ] literally translates back to the exact string sequence: **"the movie is so bad"**.  
| Yaani aapne testing ke liye model ko dubaara wahi words bheje: *"the movie is so bad"*.

### The Technical Reason for the False Positive

During training, the exact same words [1, 2, 3, 4, 5] were present in Review 1, which was very long (12 words total) and contained severe terms like *"kill myself"*. Because the dataset is incredibly small (only 5 rows) and trained for just 5 epochs, the SimpleRNN layer failed to learn that token 5 ("bad") carries negative weight. Instead, it noticed that tokens 1, 2, 3, and 4 frequently appear in the **Positive Reviews** (Reviews 2 and 3 both start with [1, 2...]). The model simply calculated a mathematical average, and the strong positive bias of tokens 1, 2, and 3 completely overwhelmed the weak negative impact of token 5.

**Asli Wajah:** Kyunki hamara data sirf 5 lines ka hai, model ne dekha ki token 1 ("the"), token 2 ("movie"), aur token 3 ("is") positive sentences mein bhi baar-baar aa rahe hain. Model ko 'bad' ka alag se matlab seekhne ka mauka hi nahi mila kyunki epoch sirf 5 the. Usne pure sentence ka average nikala, aur tokens 1, 2, 3 ke high positive weight ne token 5 ('bad') ke asar ko daba diya, jisse final answer 0.6269 (Positive) aa gaya.

## 5. Recommended Framework Optimization Plan

To scale this technical configuration to a stable pipeline capable of accurate binary text classification, apply these structural changes:

Agar aap chahte hain ki yeh code bilkul professional aur accurate kaam kare, toh ye do major corrections apply karein:

1. **Fix the Embedding Dimension Bound:** Change `input_length=100` to match your actual padded length: `model.add(Embedding(input_dim=120, output_dim=8, input_length=12))`.
2. **Upgrade to an LSTM Architecture:** Replace the standard RNN layer to resolve vanishing gradient drops:

```
from tensorflow.keras.layers import LSTM
model.add(LSTM(units=32, return_sequences=False))
```